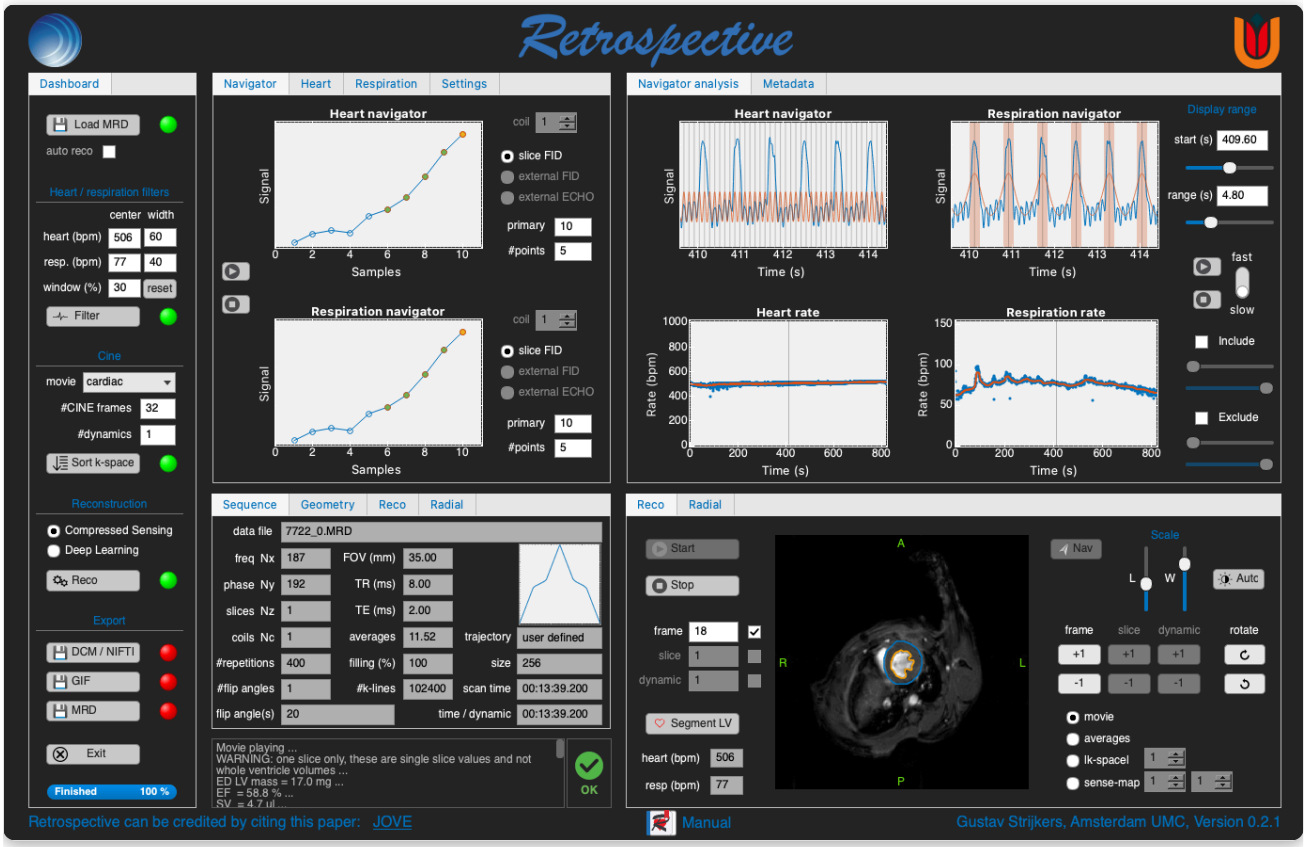


Table of contents

1. Overview
2. Contact
3. Disclaimer
4. Citing the use of Retrospective
5. Installation
6. Running the software
7. Basic operation
8. Automatic reconstruction
9. Sounds and colors
10. Credits

1. Overview

Retrospective is an application with a graphical user interface to reconstruct self-gated cardiac or respiratory CINE MRI acquired with an MR Solutions preclinical MRI system. Reconstruction is based on compressed sensing, or on a trained deep learning network, after retrospective sorting of the data into cardiac and/or respiratory time frames, using a navigator signal for cardiac and respiratory synchronization. The left ventricle can be segmented automatically in the reconstructed short-axis CINE images.

Retrospective is a stand-alone application: everything it uses is inside it, and nothing else has to be installed.

2. Contact

Retrospective and this manual are written by:

Gustav Strijkers
Amsterdam UMC
Biomedical Engineering and Physics
Amsterdam, the Netherlands
g.j.strijkers@amsterdamumc.nl

3. Disclaimer

This is a beta version. The software has been extensively tested using mouse data acquired with an MR Solutions preclinical 7.0T/24cm system. However, this does not warrant that the functions contained in the app will meet your requirements or that the operation of the app will be uninterrupted or error-free. In case of questions or issues, please contact Gustav Strijkers.

The Gibbs ringing suppression (after Kellner et al.) and the radial density compensation (after Zwart et al.) are included for research use.

4. Citing the use of Retrospective

Retrospective is free of use. However, proper referencing of the use of Retrospective in scientific publications is highly appreciated. Citations to Retrospective can be done using the following reference (link):

Daal, M.R.R., Strijkers, G.J., Calcagno, C., Garipov, R.R., Wüst, R.C.I., Hautemann, D., Coolen, B.F. Quantification of Mouse Heart Left Ventricular Function, Myocardial Strain, and Hemodynamic Forces by Cardiovascular Magnetic Resonance Imaging. J. Vis. Exp. (171), e62595, doi:10.3791/62595 (2021).

The JOVE link at the bottom of the window opens the paper.

5. Installation

macOS

Requirements: a Mac with Apple silicon (M1 or later), macOS 26 or later.

Retrospective comes as a disk image, `Retrospective-0.3.5-macOS-arm64.dmg`. Nothing else has to be installed: Python and everything the app uses are inside it.

1. Open the disk image and drag **Retrospective** onto the Applications folder in its window.
2. Open it from Applications. The app is not signed with an Apple Developer ID, so the first time macOS refuses it ("Apple could not verify ..."); click **Done**. Then open **System Settings > Privacy & Security**, scroll down to the line saying Retrospective was blocked, click **Open Anyway** and confirm. From then on it opens as any other app.

   Instead of step 2, the same in Terminal, once:

   ```
   xattr -dr com.apple.quarantine /Applications/Retrospective.app
   ```

The first start can take a little longer: macOS checks the app once.

Windows

Requirements: a PC with Windows 10 or 11, 64-bit.

Retrospective comes as an installer, `Retrospective-0.3.5-setup.exe`. Nothing else has to be installed: Python and everything the app uses are inside it.

1. Run the installer. It is not signed, so the first time Windows may stop it ("Windows protected your PC"): click **More info** and then **Run anyway**.
2. The installer puts the app in `%LOCALAPPDATA%\Programs\Retrospective` for the current user, without administrator rights (installing it for all users is an option), with a Start-menu entry and, if you choose, a desktop shortcut. At the end it can start the app.

To remove the app, use **Uninstall Retrospective** in the Start menu, or **Settings > Apps**.

The deep learning reconstruction: the first start (macOS)

The deep learning reconstruction runs fastest on the Apple GPU, with PyTorch, in a Python environment of the app's own: the folder `venv_retropy` in your home folder. When it is not there, the app asks at its start, before its window opens, whether to set it up (**Create python environment**: **Create** / **Cancel**). On **Create** a window shows how far the setup has gotten: finding a Python, making the environment, downloading the packages, installing them and checking PyTorch. It is a few hundred megabytes to download and takes a few minutes; it happens once.

You do not need a Python of your own: when the computer has none the app can use, it installs one for itself in the folder `.retropy-python` in your home folder. None of this needs administrator rights, and nothing else on the computer is touched. A log of the setup is kept in `retropyPythonSetup.log` in the temporary folder.

Cancel the setup, or answer **Cancel**, and the deep learning reconstruction runs inside the app on the processor instead, which takes longer (about 3–4 times). The app asks again at its next start.

On Windows nothing has to be set up: the deep learning reconstruction runs inside the app, on the graphics card when it can (any DirectX 12 card), else on the processor.

The LV segmentation

The LV segmentation runs inside the app on every computer: nothing to install. It uses the graphics card when there is one, else the processor.

BART (optional)

With BART, the Berkeley Advanced Reconstruction Toolbox (https://mrirecon.github.io/bart/), the compressed sensing reconstruction runs with BART; without it, with the app's own engine, based on sigpy. Both give practically the same images (within a few percent); BART is faster, especially with GPU support.

At its start the app says in its message box which one it uses: "BART toolbox detected ..." or "WARNING: BART toolbox not available, the sigpy engine is used ...", and whether BART was compiled with GPU support. Retrospective is tested with BART 1.0.00.

Installing BART on macOS with MacPorts

1. Install the Xcode command line tools:

   ```
   xcode-select --install
   ```

2. Install MacPorts from https://www.macports.org/.
3. Install the packages:

   ```
   sudo port install fftw-3-single gcc14 libpng openblas flock gmake coreutils l
   ibblocksruntime
   ```

4. Download and unpack BART:

   ```
   wget https://github.com/mrirecon/bart/archive/v1.0.00.tar.gz
   tar xvfz v1.0.00.tar.gz
   cd bart-1.0.00
   ```

5. Build and install:

   ```
   export PATH=/opt/local/libexec/gnubin:$PATH
   CC=gcc-mp-14 gmake
   sudo gmake install
   ```

   The compiler name must match the gcc version installed in step 3: gcc14 gives gcc-mp-14, gcc15 gives gcc-mp-15.

6. Check that it works:

   ```
   bart version
   ```

Installing BART on macOS with Homebrew

1. Install the Xcode command line tools: `xcode-select --install`

2. Install Homebrew:

   ```
   /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
   ```

3. Install the packages BART needs:

   ```
   brew install gcc libpng fftw openblas make llvm libomp
   ```

   The Homebrew package that provides GNU make is called `make`; it installs the program under the name `gmake`.

4. Download and unpack BART as above (step 4 of MacPorts).
5. Build and install:

   ```
   CC=gcc-15 MACPORTS=0 gmake
   sudo gmake install
   ```

   The compiler name must match the version Homebrew installed; check it with `ls /opt/homebrew/bin/gcc-*`.

6. Check that it works: `bart version`

How Retrospective finds BART on macOS

In the normal case there is nothing to configure: `sudo make install` puts BART in `/usr/local/bin`, which is where the app looks first. It looks in these folders, in this order:

Folder	
`/usr/local/bin`	where BART's own `make install` puts it
`/usr/bin`	
`/opt/homebrew/bin`	Homebrew's
`/opt/local/bin`	MacPorts'

An app opened from the Finder or the Dock does not see the settings of your shell (such as `TOOLBOX_PATH` or `PATH` in your shell profile). If your `bart` is in another folder, a link to it in `/usr/local/bin` lets the app find it. In Terminal, once, with the folder your bart is in instead of `/path/to/bart` (Terminal asks for your password):

```
sudo mkdir -p /usr/local/bin
sudo ln -s /path/to/bart/bart /usr/local/bin/bart
```

Installing BART on Windows 11

On Windows, BART runs inside the Windows Subsystem for Linux (WSL); the app calls it as `wsl bart`.

1. Install WSL. Start a Windows PowerShell as administrator and run:

   ```
   wsl --install
   ```

   This enables the feature, installs WSL 2 and installs Ubuntu in one step. A restart is needed. The first time you start the Linux prompt you are asked to create a user account. On older systems, or if `wsl --install` is not recognized, see https://pureinfotech.com/install-wsl-windows-11/.

2. Update the Linux distribution (everything from here on is typed at the Linux prompt, not in PowerShell):

```
sudo apt-get update
sudo apt-get dist-upgrade
```

3. Install the packages BART needs (gcc 12 or newer):

```
sudo apt-get install make gcc libfftw3-dev liblapacke-dev libpng-dev libopenb
las-dev gfortran
```

4. Download and unpack BART:

```
wget https://github.com/mrirecon/bart/archive/v1.0.00.tar.gz
tar xvfz v1.0.00.tar.gz
cd bart-1.0.00
```

5. Build and install (for GPU support, follow the next section instead):

```
make
sudo make install
```

6. Check that it works, at the Linux prompt `bart version`, and from a Windows command prompt `wsl bart version`. Both should print a version number.

Building BART with GPU support on Windows (provisional, highly recommended)

The reconstruction is considerably faster on a GPU.

1. Install the NVIDIA driver in Windows, following https://docs.nvidia.com/cuda/wsl-user-guide/index.html. The Windows driver is what provides GPU access inside WSL: do not install a Linux display driver inside WSL.
2. Install the CUDA toolkit inside WSL. Get the current installer from https://developer.nvidia.com/cuda-downloads, choosing Linux → x86_64 → WSL-Ubuntu. For CUDA 12.8:

```
wget https://developer.download.nvidia.com/compute/cuda/12.8.0/local_installe
rs/cuda_12.8.0_570.86.10_linux.run
sudo sh cuda_12.8.0_570.86.10_linux.run
```

3. Find the compute capability of your card at https://developer.nvidia.com/cuda-gpus. BART has to be compiled for the GPU generation it will run on; a binary built for the wrong generation compiles without complaint and then fails at the first calculation.

GPU generation	Example cards	Value to use
Pascal	GTX 1060, 1080, Titan X	`sm_61`
Turing	RTX 2060 – 2080	`sm_75`
Ampere	RTX 3060 – 3090, A100	`sm_80` / `sm_86`
Ada Lovelace	RTX 4060 – 4090	`sm_89`
Blackwell	RTX 5070 – 5090	`sm_120`

4. Set the environment variables, in the shell you will build in (adjust `CUDA_BASE` to your CUDA version and `GPUARCH_FLAGS` to the value of step 3):

```
export CUDA=1
export CUDA_BASE=/usr/local/cuda-12.8
export CUDA_LIB=lib64
export GPUARCH_FLAGS="-arch=sm_61"
export PATH=$CUDA_BASE/bin:$PATH
export LD_LIBRARY_PATH=$CUDA_BASE/$CUDA_LIB:$LD_LIBRARY_PATH
```

5. Build:

```
make -j$(nproc) CUDA=1 CUDA_BASE=$CUDA_BASE CUDA_LIB=$CUDA_LIB GPUARCH_FLAGS
="$GPUARCH_FLAGS"
sudo make install
```

   If the build fails at the linking stage with an error about libstdc++ or libgcc, point the linker at the installed gcc (check the version with `ls /usr/lib/gcc/x86_64-linux-gnu/`) and build again:

```
export LDFLAGS="-L/usr/lib/gcc/x86_64-linux-gnu/13 $LDFLAGS"
```

6. Test the GPU build: `make utest_gpu`

What matters for Retrospective is that `bart` is on the PATH inside your Linux distribution; `sudo make install` takes care of that.

6. Running the software

Open **Retrospective** from the Applications folder (macOS), or from the Start menu or the desktop shortcut (Windows). The Retrospective logo appears while the app loads, and the window follows within seconds; the app then checks for BART and for the deep learning reconstruction, and "Ready" in the message box says it is done.

Newer versions. At its start the app asks GitHub (https://github.com/Moby1971/MRI-apps) whether there is a newer release of it. The message box then shows the current and the newest version and, when there is a newer one, where to download it. Without a network connection it says that the version could not be checked, and the app works as usual. `checkForUpdates` in the settings file switches the check off; `versionCheckTimeout` sets how many seconds it waits for GitHub (5).

This manual. The **Manual** button at the bottom of the window, with the app's icon on a book, opens this manual in the computer's PDF viewer.

The window has a fixed size. On a screen too small for it, the window fits the screen and can be scrolled.

Settings and log files

Retrospective keeps its files in a folder of its own, which it makes at its first start:

macOS	`~/Library/Application Support/Retrospective/0.3.5p`
Windows	`%APPDATA%\Retrospective\0.3.5p`

Settings. The settings are kept in `retroSettings.json`. The file is created with the standard values at the first start, and can be edited by hand; changes take effect at the next start. It holds, among others, the navigator and reconstruction presets, the folder where the file browser opens (`mrdImportPath`, also the folder auto reco watches), the message level, the number of log files kept, and the deep learning switch (`AImodule`). Five entries apply to the LV segmentation (step 8): `lvUseGpu`, use the graphics card when it can be used (true); `lvMyoDensity`, the myocardial density for the mass (1.05 g/mL); `lvSmoothFrameSelection`, median filter the blood pool volume curve when choosing the ED and ES frames (true; the volumes are always measured unfiltered); `lvModel`, the segmentation network (`lv_segment_v4c5.onnx`, v4c5, `heart_model`, v4c5 with the heart model, `lv_segment_v3a.onnx`, v3a, or `lv_segment.onnx`, v1; a name that is not one of the networks the app comes with falls back to v1, with a warning); `lvConsistency`, smooth the segmentation over the cardiac cycle before the labels are chosen (true). A file that cannot be read is set aside as `retroSettings-unreadable-<time>.json` and replaced by the standard values.

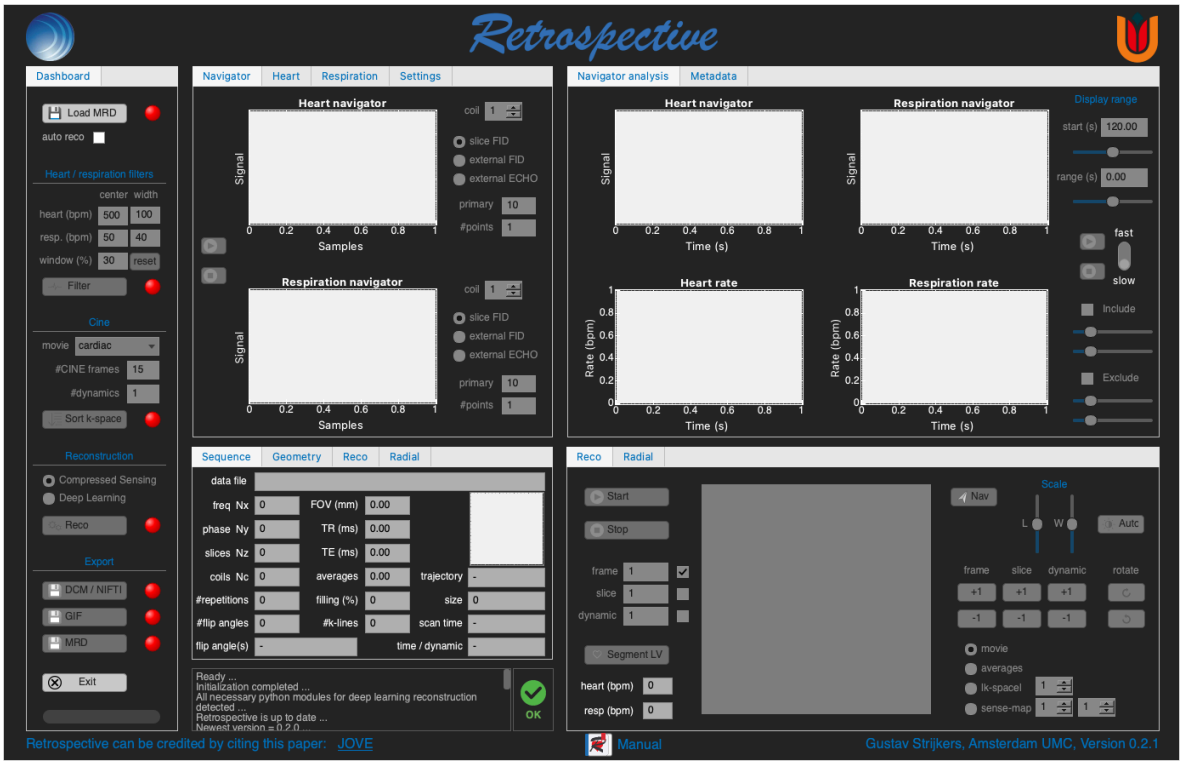
Log files. Every session writes a log file, `retrospectivepy-log-<date and time>.txt`, to the same folder. The log keeps every message, including the ones the message box does not show at the chosen message level. The 50 most recent log files are kept.

3D UTE readout profile. The readout profile for 3D UTE data, `ktraj.txt`, is in the same folder.

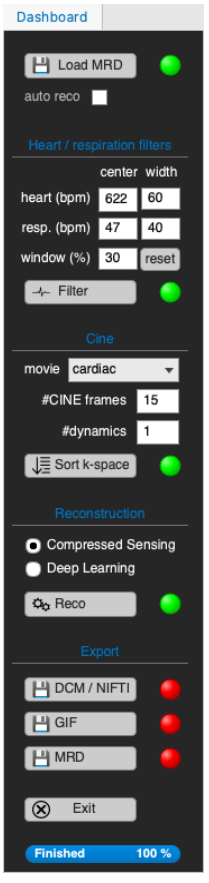
Two more logs go to the temporary folder: `retropyPythonSetup.log` (the setup of the deep learning environment, section 5) and `retropyLVseg.log` (the last LV segmentation).

7. Basic operation

The Retrospective app operates from a single window with 5 main panels.



Panel 1: Dashboard. The task buttons that control the reconstruction process. The dashboard tasks need to be completed from top to bottom. A green light next to a task indicates that it has been completed; yellow, that it is busy; red, that it has not been completed yet. Under **Load MRD** is the **auto reco** box (section 8). At the bottom of the Dashboard one progress bar shows the progress of every task: what it is doing and how far it has gotten ("Filtering", "Reconstruction 40 %", "Export 30 of 480", "Finished").



Panel 2: Navigator / Heart / Respiration / Settings. The raw navigator signal, the navigator frequency spectrum, and the navigator and app settings.

Panel 3: Navigator analysis / Metadata. The navigator signals filtered for cardiac and respiratory motion, and the heart and respiratory rates as a function of time. The **Metadata** tab beside it holds the record of the scan: the animal, the session and the contrast agent (step 9).

Panel 4: Sequence / Geometry / Reco / Radial. The acquisition and reconstruction parameters.

Panel 5: Reco / Radial. The reconstructed CINE movie. The Radial tab shows the k-space trajectory of radial, UTE and ZTE acquisitions.

Below the panels is the message box, newest message on top, with the app's status (OK, WARNING, ERROR, BUSY) beside it.

Step 1: Loading data

Press **Load MRD** to load the MRD raw data file. The data should include navigator data for cardiac and respiratory synchronization. On the scanner computer (Windows) the first dialog of a session opens in the newest exam of the scanner's MRD folder, unless the settings name a folder (`mrdImportPath`).

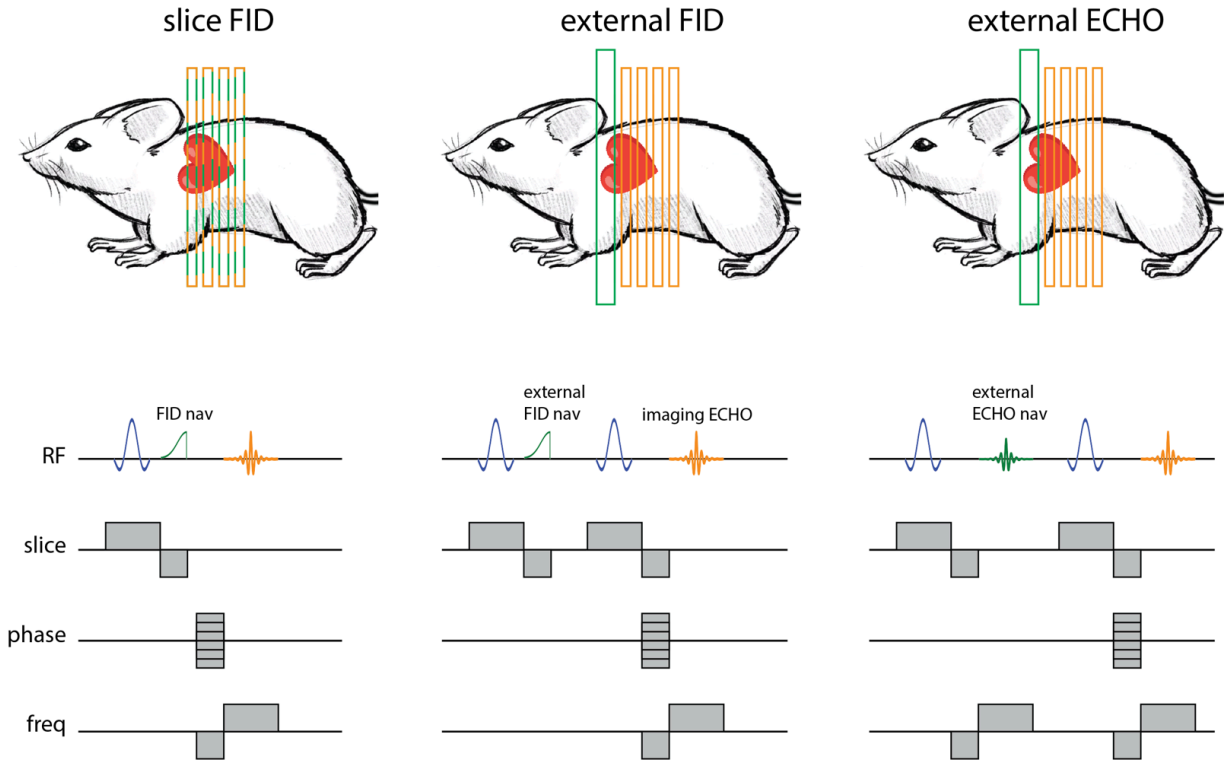
For data acquired with multiple receivers (phased-array RF coil) select one of the MRD files; the app finds the others.

The relevant acquisition parameters are shown in the Sequence tab of panel 4, including the k-space trajectory: linear, zigzag, user defined, P2ROUD (from the `.txt` file beside the MRD file), 2D radial, 3D UTE (from a `lut*.txt` file there) or 3D ZTE.

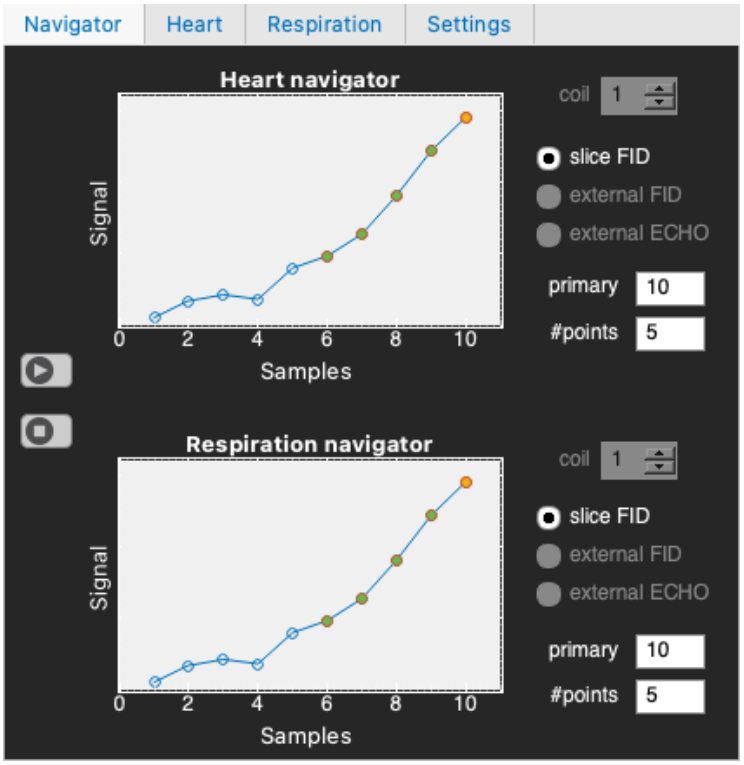
Step 2: The navigator signal

There are **3** types of navigators, which can be acquired independently and in combination, and in both 2D and 3D imaging.

- **Slice FID navigator** (left): the free-induction-decay (green) generated from the rewinder of the slice selection gradient.
- **External FID navigator** (middle): the free-induction-decay (green) produced by the rewinder of a slice selection gradient in a separate, non-spatially encoded slice.
- **External ECHO navigator** (right): the gradient-echo (green) produced in a separate slice with a readout gradient in a single dimension. The resulting navigator is the Fourier transform of the echo and therefore a 1-dimensional image (readout profile) of the object.



Depending on which type of navigator is available, one can choose slice FID, external FID or external ECHO, separately for heart and respiration, in the Navigator tab. The raw navigator signals and their spectra are shown in the Heart and Respiration tabs; the filtered signals are in panel 3 (step 4).



For multi-receiver coil data, individual navigators can be inspected with the **coil** spinner. The app picks the clearest coil or coil combination for heart and respiration.

The respiration peaks in the navigator should be positive. The app turns the navigator the right way up; the **flip** switch (Heart and Respiration tabs) overrules it when needed.

If cardiac and respiratory oscillations are not clearly visible, change the **primary** point slightly and/or increase **#points** (typically 1 to 5) in the Navigator tab, and see whether the quality improves. The play and stop buttons to the left of the plots run through the navigator samples as a movie.

The Settings tab



Presets. Three presets for navigator filtering, for mouse, rat and zebrafish data. They set the min/max heart and respiration rates, which determine the search windows for the heart and respiration rates.

freq Nx discard. The number of samples that are discarded between the end of the navigator and the beginning of the imaging echo; active when checked. If an FID artifact is visible in the reconstructed images, increase this number. This parameter is usually only relevant for FLASH sequence versions before 710.

Respiration. Images can be reconstructed during in- or expiration. The default is reconstruction during expiration, in the intervals between breaths. For reconstruction during inspiration, check **toggle in/expiration**. Discarding k-lines during respiration can be overruled by checking **ignore respiration**.

Sounds/messages. Error and warning sounds are switched on or off with **sounds on/off**. The **message level** (1 to 3) sets how much the message box shows: 1 shows the essentials, higher levels more detail. The log file keeps every message, whatever the level (section 6).

Theme. Choose a **dark** (the default) or **light** theme for the window.

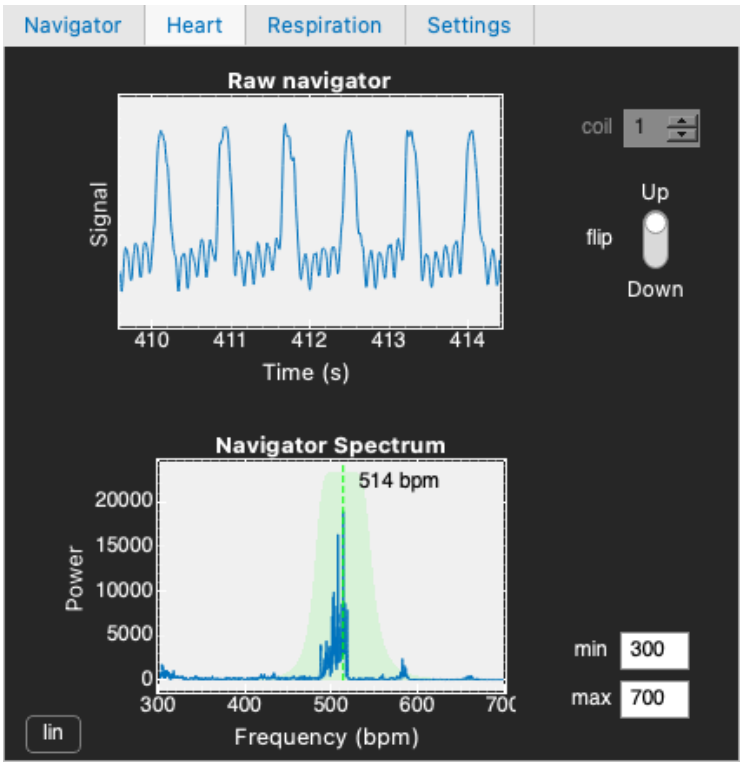
Physio filtering. filter order is the order of the Butterworth filters, and **min/max heart rate** and **min/max resp rate** bound the search for the heart and respiration rates; the presets set them. **resp shift (samples)** shifts the respiration window, to correct a delay of the respiration navigator. With **filter per slice** checked, cardiac and respiratory motion are filtered for each slice separately.

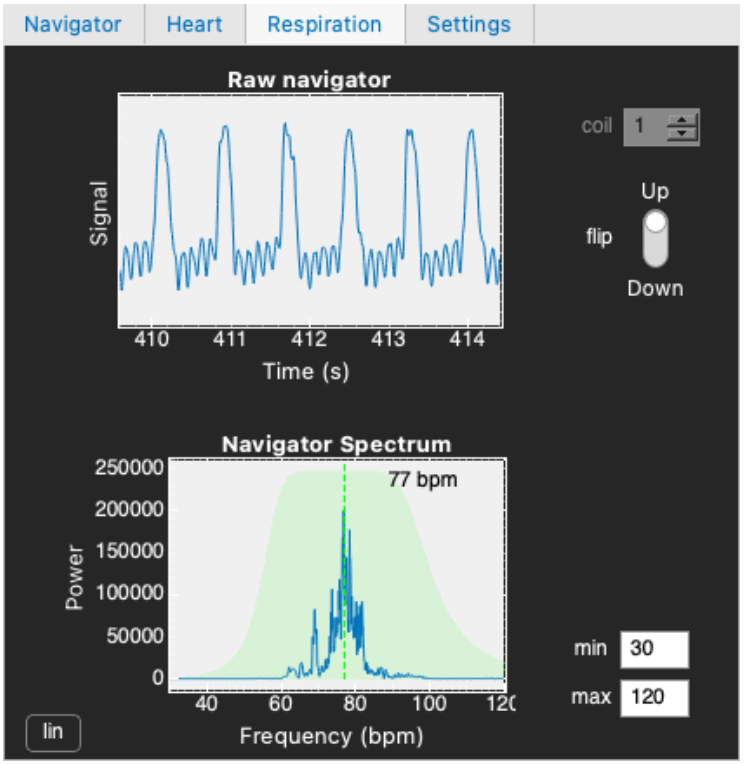
LV segmentation. model chooses the network that segments the left ventricle (step 8): **v4c5 (slice and frame context)**, the default, which also looks at each image's neighbouring slices and frames; **v4c5 + heart model**, v4c5 followed by a model of the shape and motion of the left ventricle, which keeps the base and the apex in the shape of a ventricle and gives a single long-axis slice the model's contours (about half a minute longer per stack, always with smoothing over the cycle); **v3a (short and long axis)**,

a network trained on more data, short-axis and long-axis views; or **v1 (original)**, the network of the earlier versions. The message box says which network segmented the images. **smooth over cycle** (on by default) smooths the segmentation over the cardiac cycle before the labels are chosen. Both apply from the next segmentation until the app is closed; at the start they are set from the settings file (`lvModel`, `lvConsistency`).

Step 3: The navigator power spectrum

The Heart and Respiration tabs show the raw navigator signal (top) and its power frequency spectrum (bottom). Typically, the spectrum shows peaks (and higher harmonics) at the respiratory and cardiac frequencies. The app determines the most likely heart and respiration frequencies automatically.



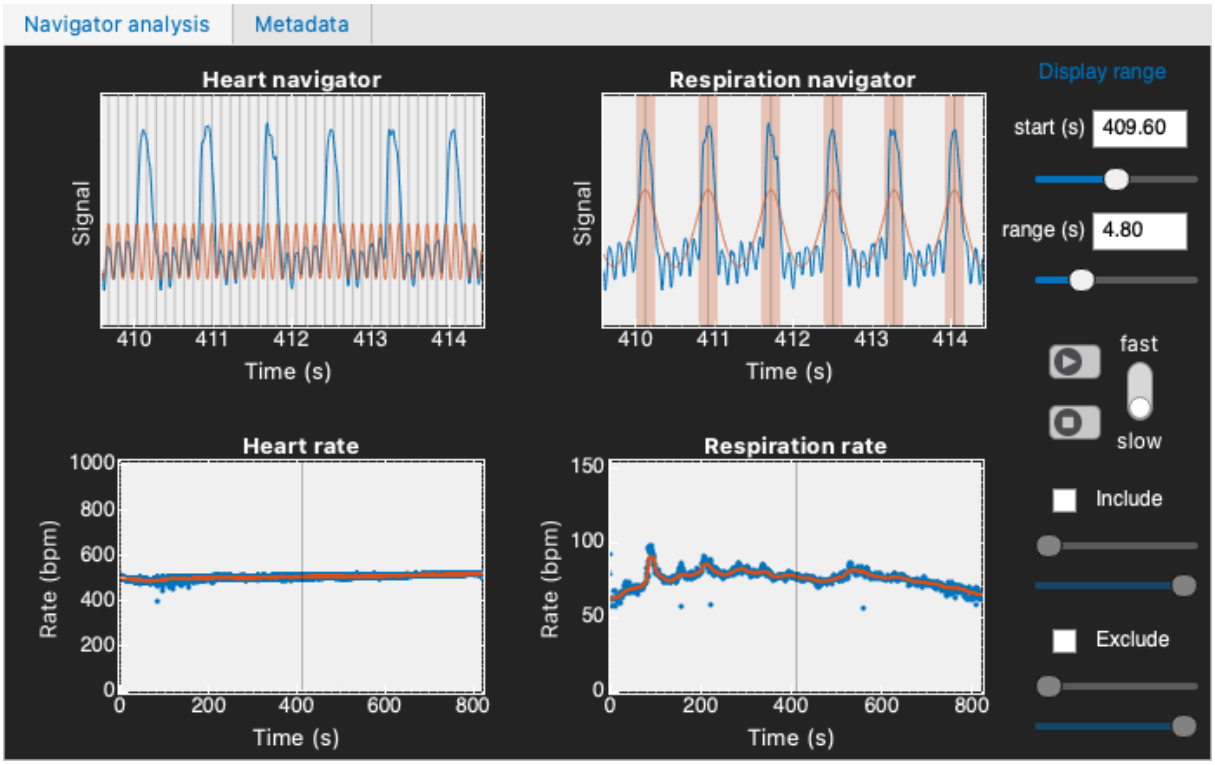


The green shaded area shows the position and width of the Butterworth filter that extracts the periodic cardiac or respiratory motion from the raw navigator. The **lin** / **log** button switches the spectrum between a linear and a logarithmic scale. **min** and **max** set the frequency range shown (bpm); they are for looking only, the search for the rates keeps its own limits (Settings tab).

When the automatic respiration and/or cardiac frequency detection fails, the **center** and **width** of the filters can be adjusted in the **Heart / respiration filters** panel of the Dashboard (**heart (bpm)**, **resp. (bpm)**). **window (%)** sets the percentage of the data that is discarded during respiratory motion. Press **reset** to reset the values.

Step 4: Filtering the navigator

Press **Filter** to filter the raw navigator signal and extract the cardiac and respiratory motion as a function of time. The result may look like this:



The top left plot shows in blue the raw navigator signal and in red the signal filtered for cardiac motion. The gray vertical lines indicate the peaks in the oscillations, which can be used to check the alignment between the red and blue signals. Bottom left is the heart rate as a function of time during the full acquisition: the blue dots are the beat-to-beat values, the red line a smoothed trend line.

The top right plot shows in blue the raw navigator signal and in red the signal filtered for respiratory motion. The shaded red areas indicate the part of the data that is discarded during respiratory motion. Bottom right is the respiration rate as a function of time; blue the breath-to-breath values, red a smoothed trend line.

Heart and respiration rates should vary smoothly over time. If they do not, adjust the Butterworth filter settings.

The **Display range** sliders and fields (**start (s)**, **range (s)**) change the part of the navigator shown, to inspect the full data set. The play and stop buttons advance the navigator automatically, **fast** or **slow**.

Including / excluding data

In case of irregular respiration or heartbeat, part of the data can be included (green) and/or excluded (red) by checking **Include** or **Exclude**, or both, and setting the range with the sliders below them. After this, press **Filter** again to filter the data with the new settings. Including or excluding data is not possible for multi-slice 2D data.

Step 5: Sorting the k-space

Choose the sorting method (**movie**), the number of frames (**#CINE frames**) and the number of dynamics (**#dynamics**) in the Cine panel of the Dashboard. Reconstruction presets are in the Reco tab of panel 4.

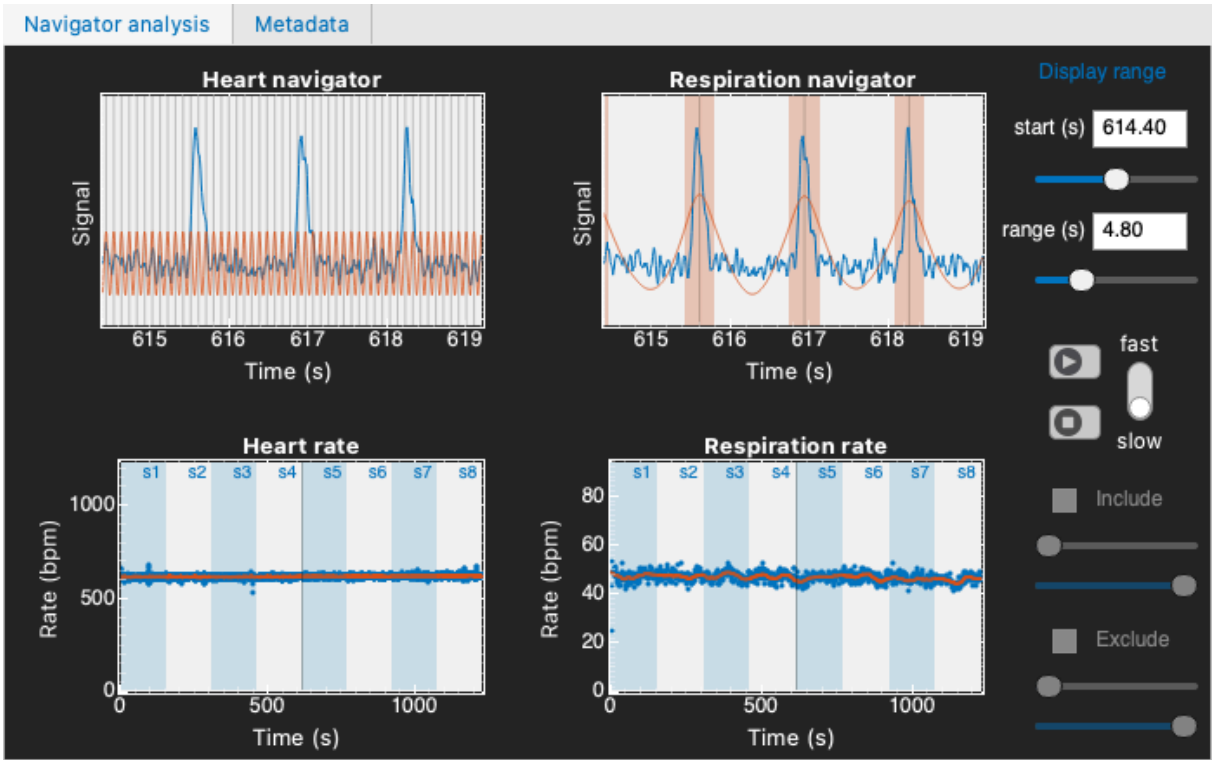
Sorting methods

1. **cardiac** — cardiac phase binning. Each heartbeat is divided into an equal number of cardiac CINE frames. The average time between frames follows from the average heart rate and the number of frames. This option is preferred for most applications, particularly for high frame-rate reconstructions for the analysis of diastolic function.
2. **cardiac abs** — cardiac absolute binning. A fixed frame time is set beforehand from the average heart rate and the number of CINE frames. This option can be more accurate for systolic function in the presence of heart-rate variations, but leads to temporal blurring in the diastolic phase.
3. **respiratory** — respiratory phase binning. Each respiration cycle is divided into an equal number of respiratory frames. **window (%)** is then not used.
4. **realtime** — the data is sorted into one CINE movie per heartbeat, for fast dynamic imaging, for example during injection of a contrast agent. This option requires a TR as short as possible and a sparse k-space trajectory that gives sufficient k-lines per frame for compressed sensing and/or parallel imaging reconstruction. The number of dynamics is set to the total number of heartbeats in the acquisition, and respiratory motion is disregarded. **#frames** under Frame sharing, in the Reco tab of panel 4, sets how many neighboring dynamics share k-lines, for better k-space filling and image quality at the expense of temporal averaging.

Press **Sort k-space** to sort the data into k-space.

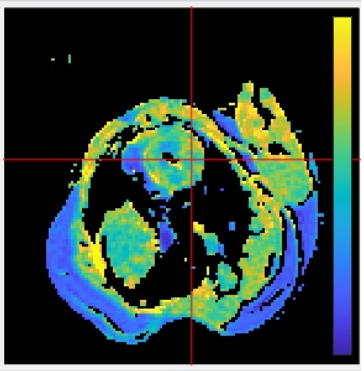
Dynamics. Data can be reconstructed as a dynamic time series, for example for dynamic contrast-enhanced acquisitions. Choose the number of dynamics, then sort and reconstruct. There is no maximum number of dynamics, but k-space becomes very sparse with a high number of dynamics, and the reconstruction may take very long. The dynamics are shown in the heart and respiration rate plots as blue/white shaded areas.

Multi-slice 2D data. For multi-slice 2D acquisitions, the heart and respiration rate plots show with blue/white shaded areas when the successive slices were acquired (s1, s2, ...). For multi-slice 2D data it is not possible to reconstruct dynamics, nor to include or exclude part of the scan.



3D data. Retrospective reconstructs 3D CINE data as well.

Variable flip-angle data for T_1 mapping. Cardiac quantitative T_1 maps can be acquired with the variable flip-angle method (Coolen et al., NMR Biomed 24, 154–162 (2011)). This requires a 3D self-gated T_1-weighted (RF and gradient spoiled) acquisition with variable flip angles (typically 5 angles, for example 2°, 5°, 10°, 15°, 20°), acquired in a single acquisition in which the flip angles are evenly distributed over the k-space repetitions. After reconstruction, export the data as a new MRD file (step 10) for T_1 mapping with the T1mapp application.



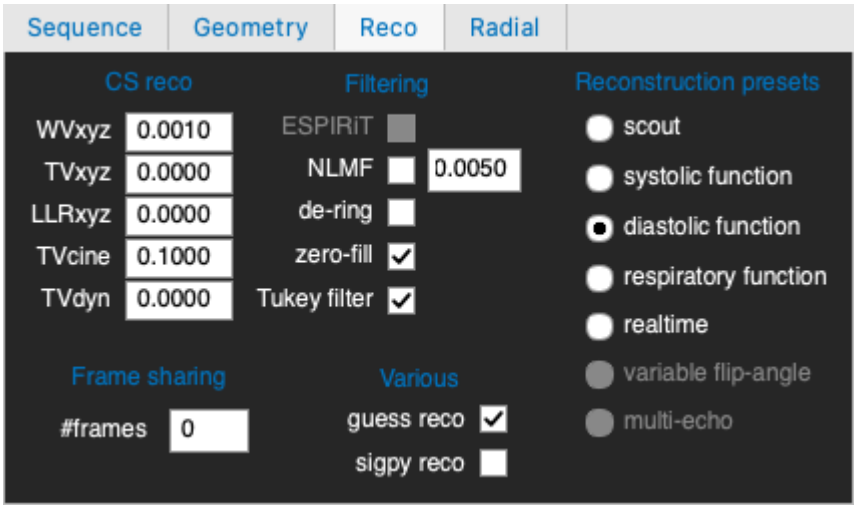
Step 6: Reconstruction

Before reconstruction, choose the reconstruction settings and regularization parameters in the Reco tab of panel 4. Two reconstruction methods are available, selected in the Reconstruction panel of the Dashboard:

- **Compressed Sensing** — the standard method, described below.
- **Deep Learning** — a trained neural network, described further down.

Press **Reco** to start the reconstruction.

A. Compressed sensing reconstruction parameters



Every parameter can be used with or without BART (section 5).

1. **WVxyz** — wavelet regularization in the spatial (x, y and z) dimensions. For the mouse cardiac CINE, a low (0.001) or zero value gives the best movie quality.
2. **TVxyz** — total variation constraint in the spatial dimensions. Higher values smooth the movie in the spatial domain. For mouse CINEs this does not seem to improve image quality; the recommended value is zero.
3. **LLRxyz** — locally low-rank regularization.
4. **TVcine** — total variation constraint in the CINE dimension. Higher values give better image quality but more temporal smoothing. For mouse CINEs, 0.1 is a good compromise.

5. **TVdyn** — total variation constraint in the dynamics dimension.
6. **ESPIRiT** — ESPIRiT reconstruction (Magn Reson Med 2014; 71(3): 990–1001) of multi-coil data, which includes GRAPPA and SENSE reconstruction.

B. Deep learning reconstruction

Select **Deep Learning** in the Reconstruction panel. The deep learning reconstruction uses DRIM, a Dynamic Recurrent Inference Machine trained on rodent cardiac CINE data.

Requirements. 2D Cartesian data, single or multi-slice, with at least 4 CINE frames; for 3D, radial, UTE and ZTE data the option is grayed out. It uses the graphics card when it can (section 5), else the processor.

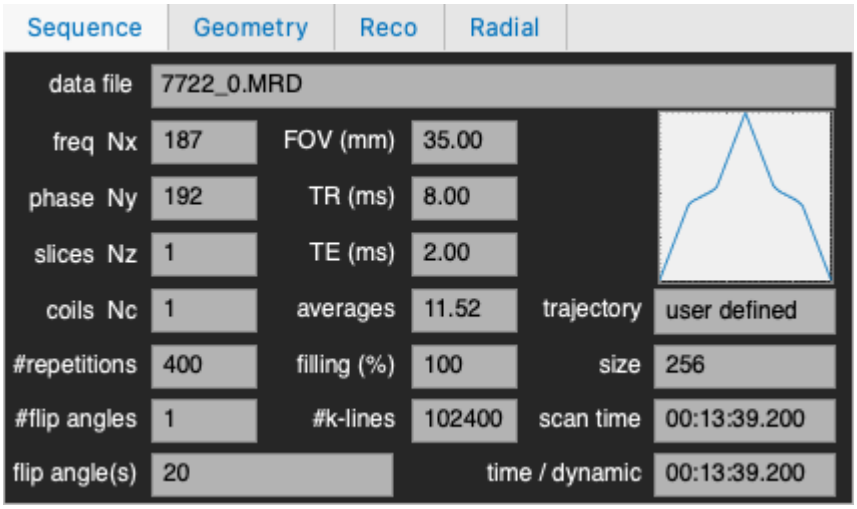
The app chooses the trained model automatically, so there is nothing to set.

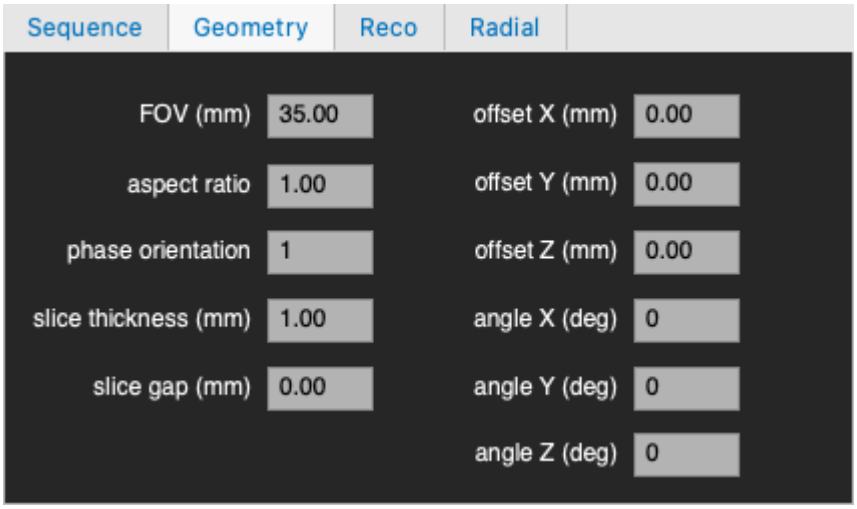
Points to be aware of

- With **zero-fill** checked in the Reco tab the images come out at 256 x 256, otherwise at the acquired matrix.
- Multi-coil and multi-dynamic data take proportionally longer.
- Memory use grows with the number of CINE frames, and the reconstruction time also with the number of slices. When a reconstruction would not fit in memory, the app does not start it, and the message box says how many frames would fit: fewer frames help, a smaller matrix does not.
- Of the settings in the Reco tab, the compressed sensing weights, ESPIRiT and **sigpy reco** do not apply to the deep learning reconstruction; **NLMF** and **de-ring** are applied after it as well.
- The message box says which device the reconstruction runs on.

The Sequence and Geometry tabs

The Sequence and Geometry tabs show the sequence parameters and the image geometry.





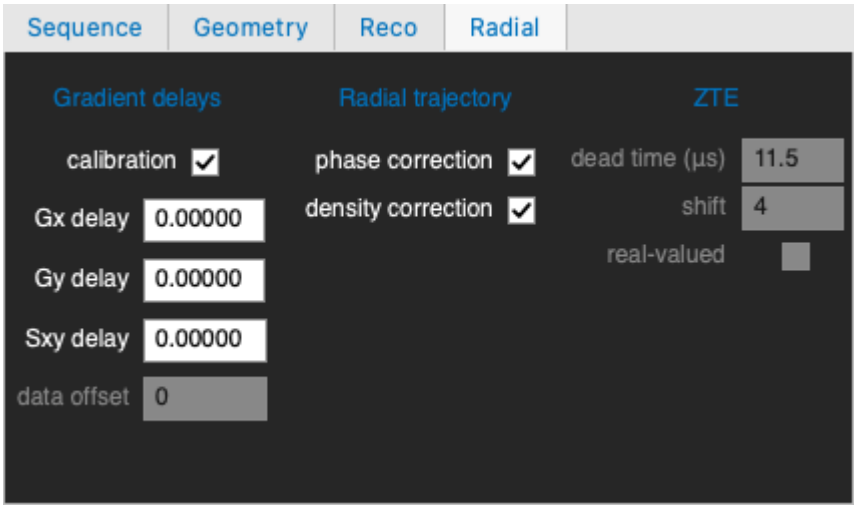
The Reco tab

The Reco tab (shown under A above) holds the reconstruction settings besides the compressed sensing weights.

- **Frame sharing** — cardiac and/or respiratory motion states can be binned with a Gaussian soft weighting of k-lines from neighboring frames (Küstner et al., NMR Biomed e4409 (2020), doi:10.1002/nbm.4409), to increase k-space filling and SNR. **#frames** sets the number of shared frames.
- **Reconstruction presets** — predetermined reconstruction settings for different applications: scout, systolic function, diastolic function, respiratory function, realtime, variable flip-angle.
- **NLMF** — a non-local means filter on the images, with the strength in the field beside it.
- **de-ring** — Gibbs ringing suppression. Checking it after sorting discards the sort; sort again. With de-ring on, the Tukey filter is not applied.
- **zero-fill** — zero-filling of k-space.
- **Tukey filter** — ellipsoid Tukey filtering of k-space.
- **guess reco** — the app makes an educated guess which type of reconstruction is appropriate; this is especially useful for automatic reconstruction.
- **sigpy reco** — reconstruct with the app's own sigpy engine instead of BART. Without BART it is checked and cannot be changed.

The Radial tab

The Radial tab holds the settings for radial, UTE and ZTE reconstructions; ZTE scans have their own **dead time (µs)**, **shift** and **real-valued**. **dead time** is the time from the center of k-space to the first sample kept; the reconstruction fills in the samples before it. Too small a value leaves solid objects dark inside with bright edges, too large a value makes them bulge in the middle. **real-valued** gives a darker background while the object keeps its signal.



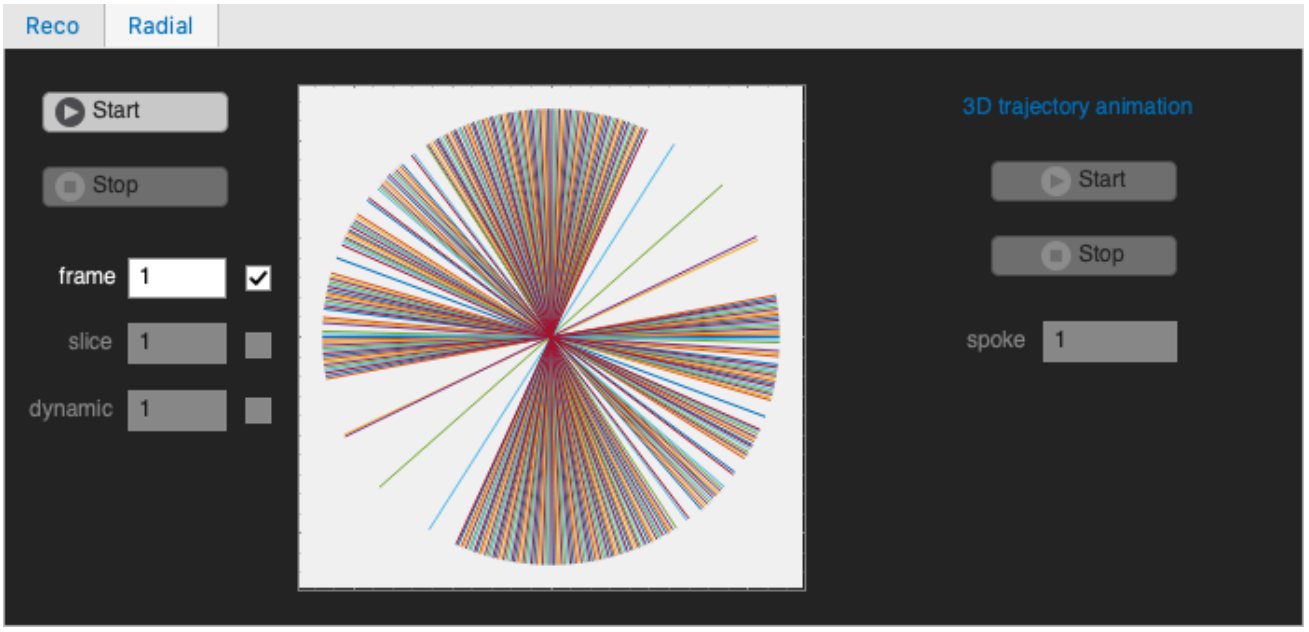
Check **calibration** to estimate the gradient delays of 2D radial data at the next reconstruction. The delays are then shown in the delay fields and the box is unchecked, so that a following reconstruction uses them; Load and Sort k-space check it again. For 3D UTE data the delays are entered in the fields.

The third delay field holds Gz for 3D UTE and the Sxy cross term for 2D radial data, and its label changes with the data. The delays, in samples, are reset when a data set is loaded; with calibration off, the delays in the fields are used as they are. For 3D UTE, **data offset** is set automatically where possible, else the setting is used.

- **phase correction** — corrects the phase of every spoke before the reconstruction.
- **density correction** — a 2D or 3D density correction in k-space for the non-uniform sampling. On by default.

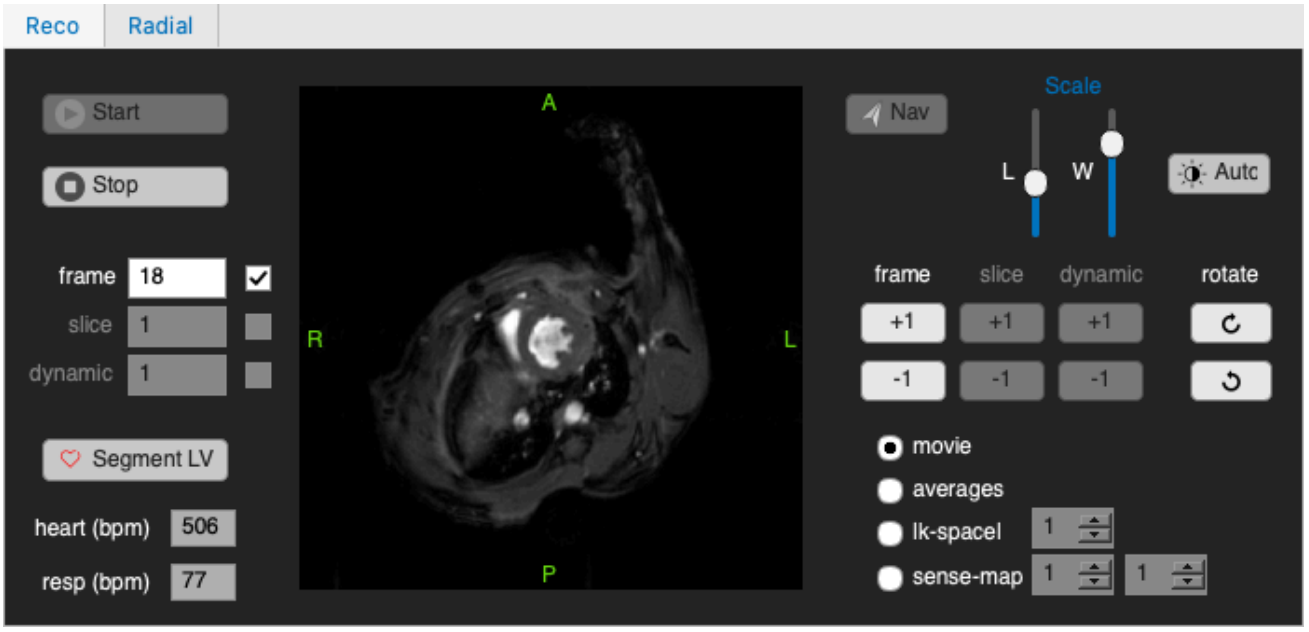
2D radial images are black outside the circle the spokes cover, in the corners of the image.

In the Radial tab of panel 5 the 2D or 3D radial k-space filling can be viewed, and the 3D radial trajectory animated (**3D trajectory animation**); the 3D view turns with the mouse.

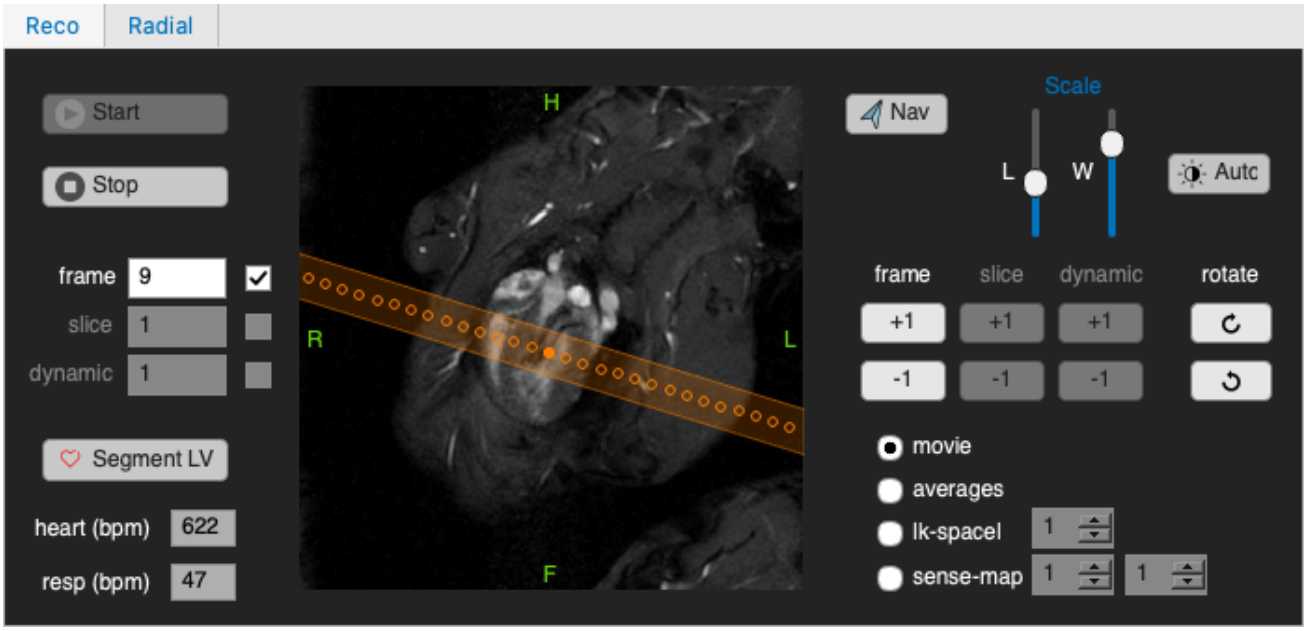


Step 7: Viewing the movie

The resulting CINE movie is shown in the Reco tab of panel 5. The radio buttons choose the **movie**, the number of **averages** per k-line and cardiac frame, the **|k-space|**, or the coil sensitivity maps (**sense-map**, ESPIRiT).



- **Start** starts the movie and **Stop** stops it.
- The boxes beside **frame**, **slice** and **dynamic** loop the movie over frames, slices and/or dynamics.
- Image intensity is adjusted with the level (**L**) and window (**W**) sliders; **Auto** sets them from the image.
- The rotate buttons turn the movie clockwise or counter-clockwise (this does not affect the DICOM and MRD export orientations).
- **+1** and **–1** below **frame** advance or rewind the movie by one time frame; below **slice** and **dynamic** they step through the slices and dynamics.
- **Nav** shows the navigator slice over the image, when it is in the plane of the current slice.
- The scroll wheel, or a box dragged over the image, zooms in; **Whole frame** shows the whole image again.



Synchronizing multi-slice 2D CINEs from one animal

Since the cardiac synchronization is not based on ECG triggering, CINEs from different slices or acquired in different orientations may not be synchronized with each other: end systole and end diastole may have different frame numbers. To correct this, first stop the movie with **Stop**. Then set the frame number to 1 in the **frame** field. Then advance or rewind the movie with **+1** or **–1** to the desired starting phase of the cardiac cycle, for example end diastole. The new starting frame is used by the DCM / NIFTI, GIF and MRD exports as well.

Step 8: LV segmentation

Press **Segment LV** to outline the left ventricle in the reconstructed short-axis CINE images and calculate the ventricular volumes.

The segmentation uses a UNet3+ neural network, fine-tuned on rodent cardiac CINE images. It labels every pixel of every frame and every slice as one of three classes:

Label	Class
0	background
1	left ventricular myocardium
2	left ventricular blood pool

Requirements. Images must be reconstructed first. Nothing else: the segmentation runs inside the app (section 5). Another network the app comes with can be chosen in the Settings tab (**model**, see "The Settings tab").

Each dynamic is treated as a cardiac cycle of its own and is segmented separately. Any matrix size works, but a field of view larger than 35 mm is cropped to the center: if the heart is not near the center of the image, part of it can be cut off. The message box warns when the field of view exceeds 35 mm.

Segmentation. Before starting the segmentation, make sure that all slices are cardiac phase aligned (see "Synchronizing multi-slice 2D CINEs from one animal" above). Then press **Segment LV**.

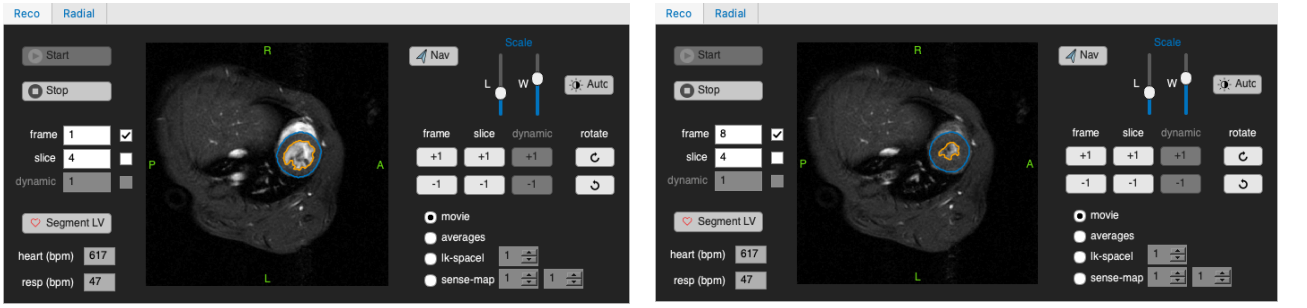
View check. The network is made for short-axis images. The message box warns when the images look like a long-axis view, or names the slices that do not look like a short-axis view (for example a slice above the base, or a heart outside the central 35 mm). The segmentation is made all the same; check its contours there.

Anatomical cleanup. Only the left ventricle is kept in each slice, and a slice where it was not found (mostly at the base and the apex) is left empty. A slice at either end of the stack whose blood pool is less than half surrounded by myocardium over most of the heart cycle, or that holds only an open arc of myocardium at end diastole, lies beyond the ventricle (above the base, for example) and is left empty in every frame. The number of slices that were cleaned or emptied is reported in the message box and written to the results file, so that it can be checked afterward.

Results

The results of the segmentation are shown over the movie in the Reco tab of panel 5: the endocardium as an orange contour, the epicardium as a blue contour.

End diastole **End systole**



An 8-slice short-axis CINE, slice 4, at end diastole (frame 1) and end systole (frame 8): EDV 58.4 µl, ESV 21.3 µl, EF 63.5 %, ED mass 113.9 mg.

The following are calculated per dynamic and shown in the message box:

EDV	end-diastolic volume (µl)
ESV	end-systolic volume (µl)
SV	stroke volume (µl)
EF	ejection fraction (%)
ED mass	left ventricular mass at end diastole (mg)

The ED and ES frames are those with the largest and smallest blood pool. The app warns when a frame chosen as ED or ES has a much larger or smaller blood pool than its neighbors, which can be an over- or under-segmentation rather than end diastole or end systole, and when only one slice was segmented (the values are then single-slice values, not whole-ventricle volumes).

Where the slices do not touch, the slice separation rather than the slice thickness is used for the volume, so that the gaps are included. The myocardial mass is calculated with a density of 1.05 g/mL. The settings for the segmentation are listed in section 6.

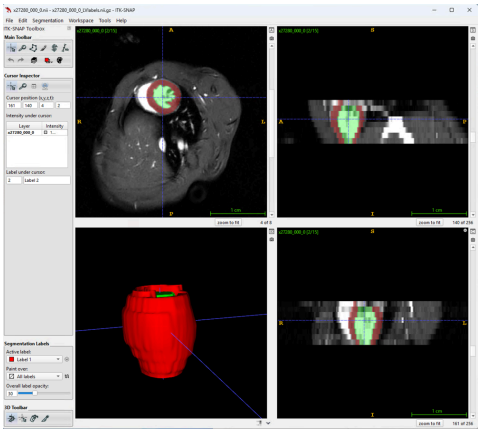
Segmentation output files

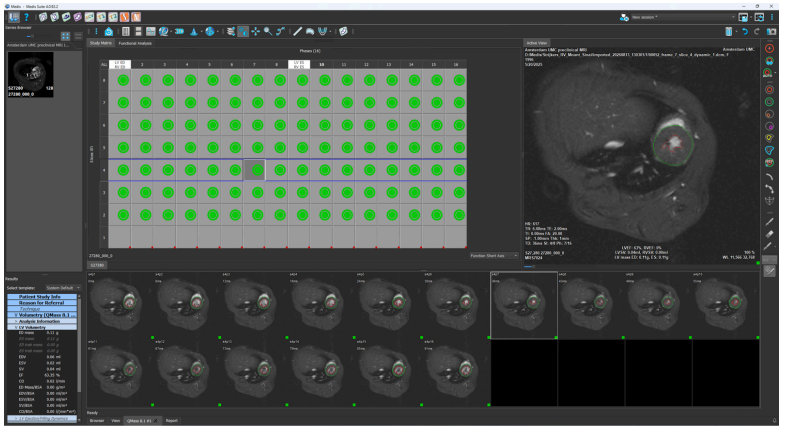
The segmentation results are written by the **DCM / NIFTI** export (step 10), together with the exported images, so that the numbers and the contours travel with the data.

1. **Volumetry** — `LVsegmentation_<tag>.xlsx`, an Excel file with two sheets.
    - *Summary*, one row per dynamic, with the columns: DateTime, Tag, Dynamic, EDframe, ESframe, Frames, Slices, EDV_ul, ESV_ul, SV_ul, EF_pct, EDmass_mg, SliceSpacing_mm, ModelPixelSpacing_mm, CleanUp, DeblobbedSlices, ClearedSlices.
    - *PerFrame*, the blood pool and myocardial volume of every frame: Dynamic, Frame, BloodPool_ul, Myocardium_ul, IsED, IsES. This is the curve from which the ED and ES frames were chosen.
2. **Contours for Medis qMASS** — `<PatientID>_STD<StudyID>_SER<n>_ACQ1_ITORIGINAL.con`, one file per dynamic, with `_dyn<number>` added to the name when there is more than one. The file contains the endocardial and epicardial contours of every frame and every slice, and can be loaded onto the exported DICOM series.
3. **ROIs for Horos / OsiriX** — `<PatientID>_STD<StudyID>_SER<n>_ACQ1_ITORIGINAL.rois_series`, the same contours in the format Horos uses for its own saved ROIs, written next to the exported DICOM files.
4. **Label maps as NIfTI** — `<name of the CINE file>_LVlabels.nii.gz`, written next to the exported NIfTI CINE, with the same header, so that it can be opened in ITK-SNAP as a segmentation of that image. The label values are the three classes listed above.

The files are written only when the exported images are the segmented ones.

ITK-SNAP **Medis**

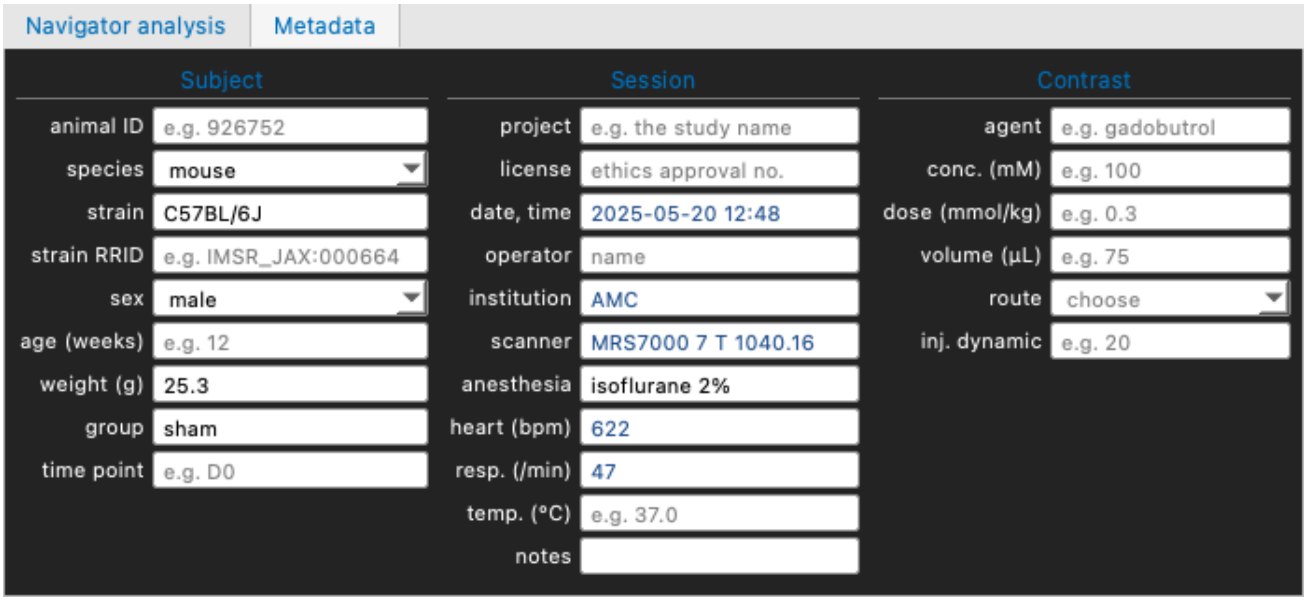




Step 9: Metadata

The **Metadata** tab in the top right panel, beside **Navigator analysis**, holds the record of the scan, which the DCM / NIFTI and GIF exports write with the movie. Its fields stand in three columns:

- **Subject:** animal ID, species, strain and its RRID, sex, age, weight, group and time point;
- **Session:** project, ethics license, date and time, operator, institution, scanner, anesthesia, heart and respiration rate, and temperature, with a field for notes;
- **Contrast:** agent, concentration, dose, volume, route and the dynamic it was injected at.



At a load the app fills in what the scan's files say: from the `.rpr` file beside the MRD file the animal ID and the time point (from the patient name typed at the console: `20240212_926752_D0` is animal 926752, day 0), the date and time, the institution and the scanner, and from the scanner's DICOM images, where they are found, the project. What the console leaves at its standard values (sex O, weight 0, the login as the operator) is left empty. After **Filter** the heart and respiration rate are the navigator's mean rates.

What was filled in for an earlier scan of the same exam is filled in again, from the records the apps wrote in the exam's `Metadata` folder (Retrospective, P2ROUD and the mapping apps alike) and from the tab as it was for the scan loaded before: the animal's details are typed only once per exam. The date, the rates, the temperature and the notes belong to one scan and are never taken from another.

A value the app filled in is shown in blue; every field can be corrected, and then counts as yours. Species, sex and route are lists to choose from, or to type in; numbers are kept with their units (a number field holding something else is edged in red). The species and sex are written with their codes (NCBI Taxonomy, PATO), so the record can be read by other software as well as by people.

Step 10: Export

There are three ways to export the movies, in the Export panel of the Dashboard.

1. **DCM / NIFTI** — exports the data in DICOM and NIfTI format for further processing in third-party software. The app searches for the scanner's DICOM images of the scan. When it finds them, the files go into a new numbered folder beside them, …`/Data/<study>/DICOM/<series>R/<n>`, and …`/Nifti/<tag>R/<n>`. When it does not, it asks for a folder, and generates the DICOM tags itself; the image position and orientation information is then lost. The DICOM files carry the species, strain, sex, age and weight of the animal where the Metadata tab has them. The export also writes the heart and respiration navigators and the cardiac triggers as a BIDS physiological recording, `<tag>_physio.tsv.gz` with `<tag>_physio.json`, next to the NIfTI files, and the LV segmentation files (step 8).
2. **GIF** — generates animated GIF movies (one per slice and dynamic for 3D and multi-slice data) of the view on screen (movie, averages or k-space), with the LV contours when segmented. Image intensity (window, level), image rotations and a zoom are preserved. The GIFs are saved in …`/Data/<study>/GIF/<tag>R/<n>`, or else in a folder you choose.

3. **MRD** — stores the data as a new MRD file in the folder of the input MRD file (or, when that folder cannot be written to, in a folder you choose): `retro_<experiments>_<echoes>_<tag>.MRD`, with `retroData_<experiments>_<echoes>_<tag>_f<filling>.h5`, an HDF5 file that holds the k-space, images, averages and parameters. If the app finds an input `.rpr` file, a new `.rpr` file is written too. The export also writes the heart and respiration trigger times and rates as text files next to the new MRD file (`cardtrigpoints.txt`, `heartrate.txt`, `resptrigpoints.txt`, `resprate.txt`, `respwindow.txt`).

Every export also writes `recoparameters_<tag>.txt`, the reconstruction settings. The DCM / NIFTI and GIF exports also write, in `…/Data/<study>/Metadata/<tag>R/<n>` (or under the folder you chose):

- `metadata.json` — the Metadata tab's record;
- `retrospective-report_<tag>.json` and `.txt` — the report of the export: the app and its libraries, the scan and a checksum of each of its files, every step of the session with its main settings, the navigator, filter, cine and reconstruction settings, the heart and respiration rates, and every file the export wrote with its checksum;
- `ro-crate-metadata.json` — the same as an RO-Crate (https://www.researchobject.org/ro-crate/), a standard description of the folder for data repositories.

The paths in them are relative to the study folder, so they hold no names of your own folders.

Step 11: Exit

Press **Exit** to close Retrospective. Closing the window does the same, also while the app is busy: the window disappears, and the app closes as soon as the running step has finished. Nothing the app started (BART, the deep learning reconstruction) keeps running after it.

8. Automatic reconstruction

When **auto reco** (under Load MRD on the Dashboard) is checked, Retrospective watches a folder and reconstructs every new MRD file automatically, for seamless integration with the MR Solutions preclinical imaging software. It watches the folder `mrdImportPath` of the settings file, or, on the scanner computer (Windows, provisional), the MRD folder of the MR Solutions software; without either, it stops.

Files already in the folder are left alone, and the newest numbered exam folder is followed. A new file is taken once it has stopped growing; files of Retrospective's or P2ROUD's own exports are ignored. Each new file is loaded, filtered, sorted, reconstructed and exported as MRD. The sounds are switched off, and the window changes color while auto reco runs.

9. Sounds and colors

Warnings and errors sound (on a Mac the system sounds Glass and Ping), unless **sounds on/off** in the Settings tab is unchecked.

The window has two themes, **dark** (the default) and **light**, chosen under **Theme** in the Settings tab. It keeps them when the computer is in dark mode.

10. Credits

- **BART toolbox and compressed sensing algorithms** — https://mrirecon.github.io/bart/. Michael Lustig, David Donoho, John M. Pauly, Sparse MRI: The application of compressed sensing for rapid MR imaging, Magn Reson Med 58, 1182–1195 (2007).
- **sigpy** — Frank Ong and Michael Lustig, SigPy: a python package for high performance iterative reconstruction, Proc. Intl. Soc. Mag. Reson. Med. 27, 4819 (2019). Used for the compressed sensing reconstruction without BART, and for the non-uniform FFT.
- **Gibbs ringing suppression** — Kellner, E., Dhital, B., Kiselev, V.G. and Reisert, M., Gibbs-ringing artifact removal based on local subvoxel-shifts. Magn Reson Med 76(5), 1574–1581 (2016).
- **Frame sharing** — Küstner et al., NMR Biomed e4409 (2020). doi:10.1002/nbm.4409.
- **Segmentation** — Shah W, Stuckey DJ, Yao T, Wrobel M, Deniszczyc E, Feng Z, Anderson S, Campbell R, Muthurangu V, Steeden J. Open-source pre-clinical image segmentation: mouse cardiac magnetic resonance imaging datasets with a deep learning segmentation framework. J Cardiovasc Magn Reson. 2026;28(1):102706. doi:10.1016/j.jocmr.2026.102706.